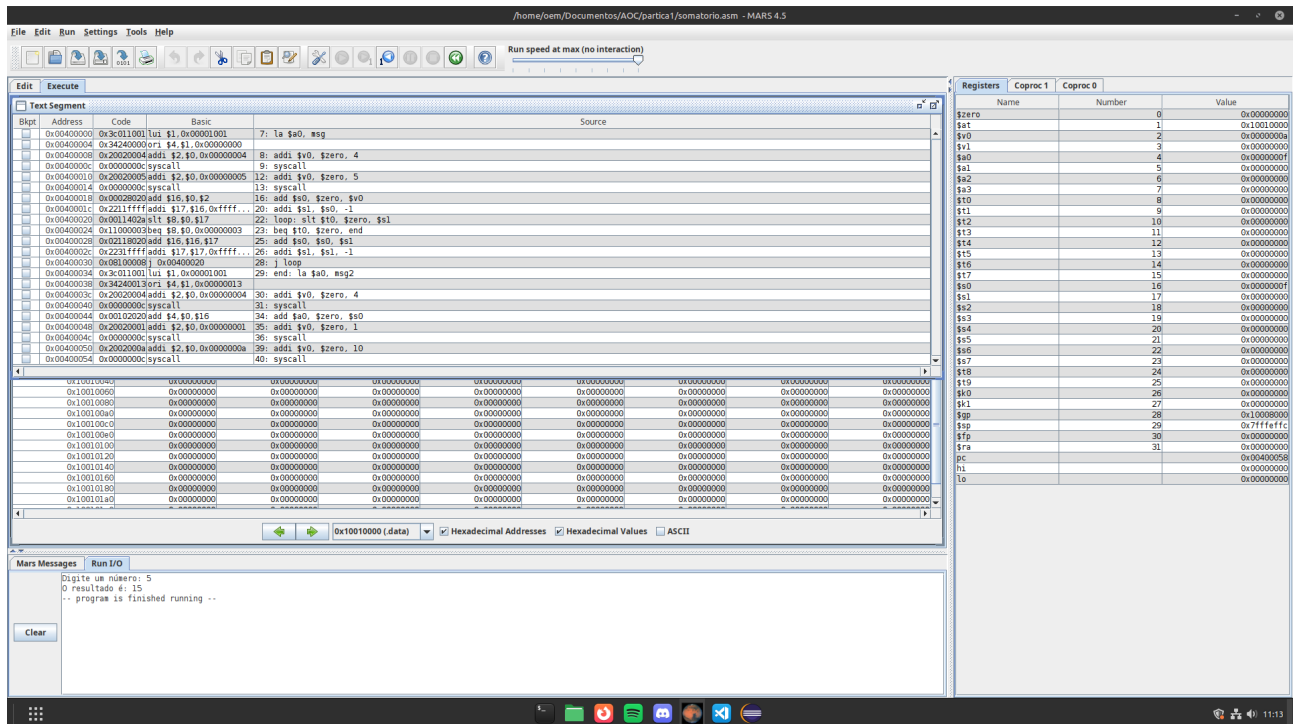


Exercício 1



Para a implementação do algoritmo de somatório, a principal decisão de projeto foi otimizar o uso de registradores realizando o cálculo em ordem decrescente. O registrador \$s0 recebe o número inserido pelo usuário e atua simultaneamente como o acumulador total da soma, evitando a necessidade de usar um registrador extra só para o resultado. O loop é controlado pelo registrador \$s1, inicializado como \$s0 – 1, que além de contador também é o próximo valor a ser somado. A estrutura de repetição e desvio foi construída com as instruções slt e beq, comparando o contador com \$zero para garantir que o programa continue somando apenas enquanto os valores decrementados forem números positivos maiores que zero.

A print evidencia o funcionamento correto da lógica implementada. No painel inferior de entrada e saída (Run I/O), constata-se que, ao inserir o valor 5, o programa retorna exatamente 15 (correspondente a $5+4+3+2+1$). Além disso, o painel lateral de registradores comprova o fluxo esperado: o registrador \$s0 finaliza a execução guardando o valor 0x0000000f (15 em hexadecimal), validando o cálculo do somatório, enquanto o registrador \$s1 encerra zerado (0x00000000), confirmando que a condição de parada do laço foi atingida com sucesso.

Exercício 2

The screenshot shows the MARS MIPS simulator interface. The main window displays assembly code with addresses, instructions, and comments. The 'Registers' panel on the right shows the state of registers \$zero through \$t0. The 'Mars Messages' panel at the bottom shows the output of the program: '1 2 4 6 8 10 12'.

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000000
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$t8	16	0x00000000
\$t9	17	0x00000000
\$s0	18	0x00000000
\$s1	19	0x00000000
\$s2	20	0x00000000
\$s3	21	0x00000000
\$s4	22	0x00000000
\$s5	23	0x00000000
\$s6	24	0x00000000
\$s7	25	0x00000000
\$s8	26	0x00000000
\$s9	27	0x00000000
\$k0	28	0x00000000
\$k1	29	0x00000000
\$k2	30	0x00000000
\$k3	31	0x00000000

Mars Messages: Run I/O

Digitar um número: 7

1 2 4 6 8 10 12

-- program is finished running --

Clear

Para a construção desta sequência, a principal decisão de código foi tratar o primeiro termo (1) como um caso especial, imprimindo-o fora do laço de repetição, já que o restante da sequência é composto exclusivamente por números pares crescentes. O registrador \$s0 foi reaproveitado como um contador regressivo do número de termos (N) solicitados pelo usuário, sendo decrementado a cada impressão. O registrador \$s1 foi inicializado com 2 e assumiu a função de armazenar os valores pares da sequência, recebendo um incremento de 2 no final de cada iteração. O laço de repetição foi controlado pela instrução condicional beq, garantindo o desvio para o fim do programa assim que o contador de termos (\$s0) chegasse a zero.

A print evidencia que o algoritmo atingiu o objetivo proposto. No painel de console (Run I/O), verifica-se que, ao inserir 7 como número de termos, o programa imprime corretamente a sequência "1 2 4 6 8 10 12". Observando o painel de registradores, a lógica de parada é confirmada: o registrador \$s0 termina a execução com o valor 0x00000000, comprovando que o contador do loop funcionou, enquanto o registrador \$s1 guarda o valor 0x0000000e (14 em decimal), que seria exatamente o próximo valor par a ser impresso caso o usuário tivesse solicitado um termo a mais.

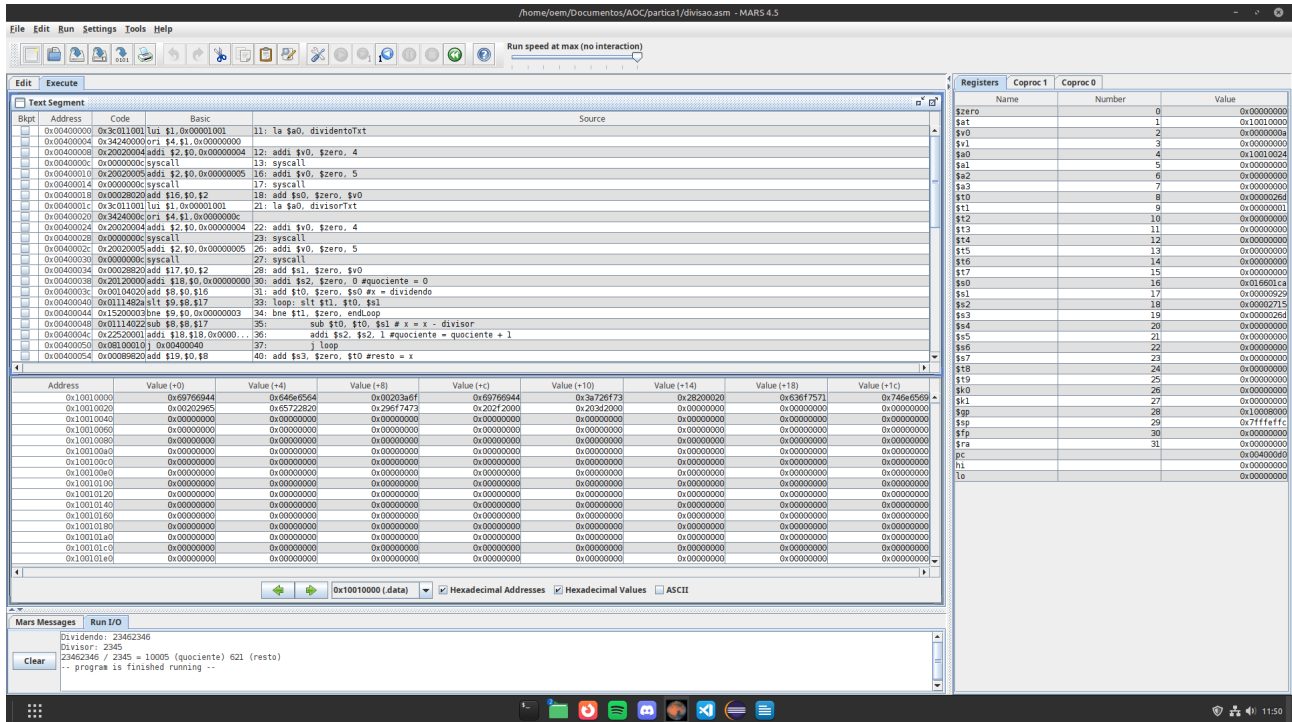
Exercício 3

The screenshot displays the MARS MIPS simulator interface. The main window shows assembly code with instructions like `la $s0, vetor`, `addi $s1, $zero, 0`, and a loop using `slt`, `beq`, and `add` instructions. The registers panel on the right shows `$s0` at 0 and `$s1` at 14. The data segment panel shows memory addresses from `0x10000000` to `0x1000000f` with values. The console panel at the bottom shows the program's output, including "Informe um número" prompts and the final message "Novos valores de V: 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40".

Para a transcrição do código de alto nível para Assembly, a principal decisão estrutural foi a estratégia para calcular o deslocamento de memória na hora de acessar o vetor declarado com `.word`. Como cada elemento ocupa 4 bytes e o conjunto de instruções do roteiro não inclui operações de multiplicação, o índice do laço (`$s1`) foi somado a si mesmo duas vezes sucessivas para simular a multiplicação por 4, garantindo o alinhamento correto dos endereços. As 2 estruturas de repetição (`for` e `while`) foram implementadas combinando as instruções `slt` e `beq` para comparar o contador com o limite de 20 termos e realizar o desvio condicional. Além disso, as atualizações dos valores no vetor foram feitas através das instruções de transferência de dados `lw` e `sw`.

A print atesta o comportamento correto da aplicação, tanto na interface quanto na memória do simulador. No painel de console (Run I/O), observa-se que o vigésimo e último número inserido no vetor foi o valor 2 (equivalente a $v[n-1]$). Como a lógica do algoritmo exige que o acumulador `x` receba esse valor a cada iteração, a saída impressa "Novos valores de V:" mostra a exata progressão aritmética esperada: 2, 4, 6, 8 até 40. Essa precisão também é evidenciada na aba Data Segment, que mostra as posições de memória do vetor sendo sobrescritas corretamente com os novos múltiplos em hexadecimal (`0x00000002`, `0x00000004`, `0x00000006`, etc.), enquanto o registrador `$s1` encerra com o valor `0x00000014` (20 em decimal), confirmando que o vetor foi percorrido em sua totalidade.

Exercício 4



Para a implementação do algoritmo da divisão utilizando apenas operações de soma e subtração, a principal decisão de projeto foi a estruturação da condição do laço while. Como o objetivo era repetir a subtração enquanto o dividendo provisório (x) fosse maior ou igual ao divisor, utilizei a instrução slt para verificar se x (\$t0) tornava-se menor que o divisor (\$s1). Em caso afirmativo, a instrução condicional bne desviava o fluxo para o fim do laço (endLoop), quebrando a repetição de forma eficiente. Dentro do laço, a instrução sub atualizava o valor de x a cada iteração, enquanto o registrador \$s2 atuava como o contador para o quociente.

A print evidencia o funcionamento correto do programa. No painel de console (Run I/O), ao testar com um dividendo alto (23462346) e um divisor de 2345, a saída formatada exibe precisamente 10005 para o quociente e 621 para o resto. O painel lateral de registradores comprova matematicamente o resultado: o registrador \$s2 (quociente) encerra com o valor 0x00002715 (10005 em decimal) e o \$s3 (resto) com 0x0000026d (621 em decimal). Além disso, o registrador temporário \$t1 finaliza com o valor 0x00000001, provando que a condição de parada projetada na instrução slt foi avaliada como verdadeira, garantindo a saída correta do loop.